

GUIDE DE REALISATION

ChatDocs OHADA · Du depot vide au deploiement, etape par etape

0. Comment lire ce guide

Ce document se lit dans l'ordre et s'exécute dans l'ordre. Chaque étape produit un résultat vérifiable avant de passer à la suivante — c'est la seule façon de ne pas se retrouver, en semaine 6, avec du code qui ne marche pas sans savoir depuis quand.

L'ordre choisi va **du plus risqué au plus confortable**. Les trois premières phases ne produisent aucun écran visible, ce qui est psychologiquement pénible quand on est développeur. Mais elles éliminent le seul vrai danger du projet : un moteur qui ne remonte pas les bons articles. Construire l'interface d'abord donnerait l'illusion d'avancer en laissant ce risque intact.

Phase	Contenu	Durée	Résultat vérifiable
A	Préparation de l'environnement	1–2 jours	Docker, Python, Node opérationnels
B	Corpus : du PDF à la base de données	1–2 semaines	Articles interrogeables en SQL
C	Recherche hybride en ligne de commande	3–5 jours	Les bons articles remontent
D	Jeu d'évaluation figé	2–3 jours	50 questions écrites et validées
E	API : le pipeline RAG complet	1–2 semaines	Endpoint /chat/question fonctionnel
F	Frontend Angular	2 semaines	Parcours principal utilisable
G	Comptes, quota, bibliothèque, historique	1 semaine	Application complète
H	Déploiement et finition	3–5 jours	Lien public + démo prête

Règle d'or : ne passe jamais à la phase suivante si le résultat vérifiable de la phase en cours n'est pas atteint. Un corpus mal découpé contamine tout ce qui vient après, et le bug sera trouvé trois semaines plus tard, dans l'interface, où il est incompréhensible.

A. Préparation de l'environnement

Phase A — 1 à 2 jours. Tout installer et vérifier avant d'écrire une ligne de code métier.

A.1 — Outils à installer

Outil	Version	Vérification
Python	3.11 ou 3.12	<code>python --version</code>
Node.js	20 LTS	<code>node --version</code>
Angular CLI	18	<code>ng version</code>
Docker Desktop	récent	<code>docker --version</code>
Git	récent	<code>git --version</code>
Tesseract OCR	5.x (+ pack français)	<code>tesseract --list-langs</code> doit afficher fra

Sur Windows : installe Tesseract via l'installateur UB-Mannheim en cochant explicitement le pack de langue française, puis ajoute son dossier au PATH. C'est la source d'erreur la plus fréquente de toute la phase B.

A.2 — Structure des dépôts

Deux dossiers dans un seul dépôt Git (monorepo simple) : c'est plus facile à gérer seul qu'un dépôt par composant, et le formateur n'a qu'un lien à ouvrir.

```
chatdocs-ohada/
  backend/
    app/
      main.py           point d'entree FastAPI
      config.py          variables d'environnement
      db.py              connexion PostgreSQL
      models.py          tables SQLAlchemy
      schemas.py         schemas Pydantic (entree/sortie API)
      routers/           auth.py . chat.py . corpus.py
      services/
        recherche.py     recherche hybride
        rag.py            orchestration de la reponse
        llm.py            interface fournisseur LLM
        securite.py       hachage, JWT
    ingestion/
      1_extraire.py      PDF -> texte brut
      2_decouper.py      texte -> articles
      3_charger.py       articles -> PostgreSQL
      4_vectoriser.py    articles -> embeddings
    tests/
      requirements.txt
  frontend/              projet Angular 18
  evaluation/
    questions.json       le jeu de 50 questions
    lancer_eval.py
  docker-compose.yml
  .env.example
  README.md
```

A.3 — Docker Compose : la base de données

PostgreSQL avec pgvector tourne dans Docker : aucune installation à faire sur ta machine, et tu obtiens exactement le même environnement que sur le serveur.

```
# docker-compose.yml
services:
  db:
    image: pgvector/pgvector:pg16
    environment:
      POSTGRES_USER: chatdocs
      POSTGRES_PASSWORD: chatdocs_local
      POSTGRES_DB: chatdocs
    ports: ["5432:5432"]
    volumes: ["pgdata:/var/lib/postgresql/data"]
volumes:
  pgdata:

docker compose up -d
docker compose exec db psql -U chatdocs -d chatdocs -c "CREATE EXTENSION IF NOT EXISTS vector;"
# doit repondre : CREATE EXTENSION
```

A.4 — Environnement Python

```
cd backend
python -m venv .venv
# Windows : .venv\Scripts\activate | Linux/Mac : source .venv/bin/activate
pip install fastapi uvicorn[standard] sqlalchemy psycpg2-binary pgvector \
    pydantic-settings python-jose[cryptography] passlib[bcrypt] \
    pdfplumber pytesseract pillow httpx pytest python-multipart
pip freeze > requirements.txt
```

A.5 — Variables d'environnement

```
# .env.example (a copier en .env — .env est dans .gitignore)
DATABASE_URL=postgresql://chatdocs:chatdocs_local@localhost:5432/chatdocs
LLM_API_KEY=votre_cle_ici
LLM_MODELE=nom_du_modele
```

```
JWT_SECRET=chaîne_aleatoire_longue
JWT_EXPIRATION_MINUTES=30
SEUIL_PERTINENCE=0.55
NB_ARTICLES_CONTEXTE=8
```

Ne commite jamais le fichier .env. Vérifie que .gitignore contient .env, .env/, node_modules/ et __pycache__/ avant le premier commit. Une clé d'API publiée sur GitHub est révoquée en quelques heures — et parfois utilisée avant.

A.6 — Résultat vérifiable de la phase A

- `docker compose up -d` démarre la base sans erreur.
- L'extension vector est créée.
- `uvicorn app.main:app --reload` démarre et /docs s'ouvre dans le navigateur.
- `ng serve` affiche la page Angular par défaut.
- Le dépôt est sur GitHub avec les branches main, formateur, apprenant.

B. Le corpus : du PDF officiel à la base de données

Phase B — 1 à 2 semaines. La phase la plus longue et la plus déterminante du projet.

Commence par UN SEUL texte. L'AUSCGIE (droit des sociétés) est le meilleur choix : structure régulière, numérotation propre, questions fréquentes. Tu élargiras au CGI seulement une fois toute la chaîne validée de bout en bout. Vouloir ingérer quatre textes d'emblée est l'erreur qui fait échouer ce type de projet.

B.1 — Collecte et traçabilité

Télécharge le PDF depuis la source officielle. Avant toute chose, enregistre sa fiche de provenance : URL, date de téléchargement, empreinte SHA-256 du fichier. Cette fiche est ce qui te permettra, dans six mois, de savoir exactement quelle version tu as ingérée.

```
import hashlib, json, datetime
chemin = "sources/auscgie_2014.pdf"
sha = hashlib.sha256(open(chemin, 'rb').read()).hexdigest()
json.dump({
    "fichier": chemin,
    "url_source": "https://...", # URL officielle exacte
    "telecharge_le": datetime.date.today().isoformat(),
    "sha256": sha
}, open("sources/auscgie_2014.provenance.json", "w"), indent=2, ensure_ascii=False)
```

B.2 — Extraction du texte

Deux cas. Si le PDF contient du texte natif, pdfplumber suffit et le résultat est propre. Si le PDF est un scan, il faut passer par l'OCR — plus lent et plus risqué.

```
# ingestion/1_extraire.py
import pdfplumber, pytesseract
from pdf2image import convert_from_path
def extraire(chemin_pdf, sortie_txt):
    pages = []
    with pdfplumber.open(chemin_pdf) as pdf:
        for i, page in enumerate(pdf.pages, 1):
            t = page.extract_text() or ""
            pages.append((i, t))
    # detection : moins de 100 caracteres par page en moyenne => PDF scanne
    moyenne = sum(len(t) for _, t in pages) / max(len(pages), 1)
    if moyenne < 100:
        print("PDF scanne detecte -> OCR (long)")
        images = convert_from_path(chemin_pdf, dpi=300)
        pages = [(i, pytesseract.image_to_string(img, lang='fra'))
                  for i, img in enumerate(images, 1)]
    with open(sortie_txt, "w", encoding="utf-8") as f:
        for num, texte in pages:
            f.write(f"\n===PAGE {num}===\n{texte}")
```

```
return pages
```

Après un OCR, relis toujours un échantillon à l'œil. L'OCR confond régulièrement 0/O, 1/I, 5/S — or dans un texte juridique, un numéro d'article ou un taux erroné est bien pire qu'une absence d'information. Si l'OCR est trop sale, cherche une meilleure source plutôt que de corriger à la main.

B.3 — Découpage en articles

C'est l'étape centrale de tout le projet. On parcourt le texte ligne à ligne en maintenant un « état » de la hiérarchie courante (livre, titre, chapitre, section), et dès qu'on rencontre un en-tête d'article, on clôt l'article précédent et on ouvre un nouveau.

```
# ingestion/2_decouper.py
import re, json
RE_ARTICLE = re.compile(r"^\\s*Article\\s+(\\d+(?:\\s*(?:bis|ter|quater)))?\\s*[\\.\\-:~]?", re.I)
RE_NIVEAUX = [
    ("livre", re.compile(r"^\\s*LIVRE\\s+([IVXLC]+|\\d+)\\s*[:\\.\\-]?\\s*(.*)$", re.I)),
    ("titre", re.compile(r"^\\s*TITRE\\s+([IVXLC]+|\\d+)\\s*[:\\.\\-]?\\s*(.*)$", re.I)),
    ("chapitre", re.compile(r"^\\s*CHAPITRE\\s+([IVXLC]+|\\d+)\\s*[:\\.\\-]?\\s*(.*)$", re.I)),
    ("section", re.compile(r"^\\s*SECTION\\s+([IVXLC]+|\\d+)\\s*[:\\.\\-]?\\s*(.*)$", re.I)),
]
ORDRE = ["livre", "titre", "chapitre", "section"]
def decouper(chemin_txt):
    contexte, articles, courant = {}, [], None
    for ligne in open(chemin_txt, encoding="utf-8"):
        brut = ligne.rstrip()
        # 1) un en-tete de niveau ? on reinitialise les niveaux inferieurs
        entete = False
        for nom, regex in RE_NIVEAUX:
            m = regex.match(brut)
            if m:
                contexte[nom] = f"{nom.capitalize()} {m.group(1)}"
                for inferieur in ORDRE[ORDRE.index(nom)+1:]:
                    contexte.pop(inferieur, None)
                entete = True
                break
        if entete:
            continue
        # 2) un debut d'article ?
        m = RE_ARTICLE.match(brut)
        if m:
            if courant:
                articles.append(courant)
            courant = {
                "numero": m.group(1).replace(" ", " ").strip(),
                "chemin": " > ".join(contexte[n] for n in ORDRE if n in contexte),
                "contenu": brut[m.end():].strip()
            }
        elif courant and brut.strip() and not brut.startswith("===PAGE"):
            courant["contenu"] += " " + brut.strip()
        if courant:
            articles.append(courant)
    return articles
```

B.4 — Contrôles automatiques obligatoires

Avant de charger quoi que ce soit en base, fais tourner ces contrôles. Ils détectent 90 % des erreurs de découpage en quelques secondes.

```
def controler(articles):
    problemes = []
    # a) numerotation : reperer les trous et les doublons
    nums = [a["numero"] for a in articles]
    if len(nums) != len(set(nums)):
        doublons = {n for n in nums if nums.count(n) > 1}
        problemes.append(f"Numeros en double : {sorted(doublons)}")
        entiers = sorted(int(re.match(r"\\d+", n).group()) for n in nums)
        trous = [i for i in range(entiers[0], entiers[-1]) if i not in entiers]
        if trous:
            problemes.append(f"Articles manquants : {trous[:20]}")
    # b) contenus vides ou aberrants
    for a in articles:
```

```

if len(a["contenu"]) < 40:
    problemes.append(f"Article {a['numero']} trop court ({len(a['contenu'])} car.)")
if len(a["contenu"]) > 12000:
    problemes.append(f"Article {a['numero']} trop long -> decoupage rate ?")
if not a["chemin"]:
    problemes.append(f"Article {a['numero']} sans chemin hierarchique")
return problemes

```

Un article « trop long » signale presque toujours un découpage raté : l'expression régulière n'a pas reconnu le début de l'article suivant, et deux articles ont fusionné. C'est le défaut le plus sournois, parce que la recherche remontera un bloc contenant la bonne réponse noyée dans autre chose.

B.5 — Relecture humaine

Les contrôles automatiques ne remplacent pas les yeux. Ouvrez 20 articles au hasard et comparez-les au PDF original : le numéro, le chemin hiérarchique et le contenu complet. Vérifiez systématiquement le premier article, le dernier, et trois articles pris à des endroits différents du texte. **C'est ce quart d'heure qui fait la différence entre un corpus fiable et un corpus qui a l'air fiable.**

B.6 — Chargement en base

```

# ingestion/3_charger.py (extrait)
from sqlalchemy import create_engine, text
import os, json
moteur = create_engine(os.environ["DATABASE_URL"])
def charger(sigle, titre, version, date_conso, articles, provenance):
    with moteur.begin() as cx:
        texte_id = cx.execute(text("""
            INSERT INTO texte (sigle,titre,type,version,date_consolidation,
                               source_url,source_sha256,valide_par)
            VALUES (:s,:t,'acte_uniforme',:v,:d,:u,:h,:p)
            RETURNING id"""),
            {"s":sigle,"t":titre,"v":version,"d":date_conso,
             "u":provenance["url_source"],"h":provenance["sha256"],"p":"Christian"}
        ).scalar())
    for a in articles:
        cx.execute(text("""
            INSERT INTO article (texte_id,numero,chemin,contenu,
                               date_entree_vigueur,recherche_fts)
            VALUES (:tid,:n,:c,:co,:d,
                    to_tsvector('french', :co))"""),
            {"tid":texte_id,"n":a["numero"],"c":a["chemin"],
             "co":a["contenu"],"d":date_conso})
    return texte_id

```

B.7 — Génération des embeddings

Chaque article est vectorisé **avec son chemin hiérarchique en préfixe**. Un article isolé est souvent incompréhensible ; « Livre I > Titre II > Des assemblées générales — Article 337 : ... » porte beaucoup plus de sens qu'« Article 337 : ... ». Ce détail améliore nettement la pertinence.

```

# ingestion/4_vectoriser.py (extrait)
def texte_a_vectoriser(article):
    return f"{article['sigle']} > {article['chemin']} > Article {article['numero']}\n{article['contenu']}"
# traiter par lots de 50 à 100 pour limiter le nombre d'appels
# stocker le vecteur dans article.embedding (type VECTOR)
# puis créer l'index :
# CREATE INDEX idx_article_vec ON article
# USING hnsw (embedding vector_cosine_ops);

```

Crée l'index HNSW APRÈS avoir inséré tous les vecteurs, jamais avant : la construction de l'index sur une table déjà remplie est bien plus rapide, et l'index se maintient ensuite tout seul.

B.8 — Résultat vérifiable de la phase B

- Le nombre d'articles en base correspond au nombre d'articles du PDF (compte manuel de contrôle).
- Les contrôles automatiques ne remontent aucun problème bloquant.
- 20 articles vérifiés à la main correspondent exactement à l'original.

- Chaque article a un chemin hiérarchique non vide et un embedding non nul.
- La table `texte` contient la provenance complète (URL, SHA-256, validateur).

C. Recherche hybride en ligne de commande

Phase C — 3 à 5 jours. Sans interface, sans LLM. On vérifie que les bons articles remontent.

Cette phase est la plus importante du projet et celle qu'on est le plus tenté de sauter. Si la recherche ne remonte pas les bons articles, aucun modèle de langage au monde ne produira une bonne réponse : il répondra brillamment à partir des mauvais textes.

C.1 — Pourquoi hybride et pas seulement vectoriel

La recherche vectorielle comprend le sens (« délai de convocation » trouve un article qui parle de « quinze jours avant la réunion »), mais elle est mauvaise sur les termes exacts : un numéro d'article, un taux, un sigle. La recherche lexicale fait l'inverse. On combine les deux.

```
-- recherche vectorielle (pgvector, distance cosinus)
SELECT id, numero, chemin, contenu,
       1 - (embedding <=> :vecteur_question) AS score
FROM article
WHERE date_abrogation IS NULL
ORDER BY embedding <=> :vecteur_question
LIMIT 20;

-- recherche lexicale (full-text francais)
SELECT id, numero, chemin, contenu,
       ts_rank(recherche_fts, plainto_tsquery('french', :question)) AS score
FROM article
WHERE date_abrogation IS NULL
      AND recherche_fts @@ plainto_tsquery('french', :question)
ORDER BY score DESC
LIMIT 20;
```

C.2 — Fusion des deux listes (RRF)

La fusion par rang réciproque (Reciprocal Rank Fusion) évite d'avoir à normaliser des scores qui n'ont pas la même échelle. On ne regarde que la **position** de chaque article dans chaque liste.

```
def fusion_rrf(liste_vect, liste_lex, k=60, n=8):
    scores = {}
    for rang, art in enumerate(liste_vect, 1):
        scores[art["id"]] = scores.get(art["id"], 0) + 1.0 / (k + rang)
    for rang, art in enumerate(liste_lex, 1):
        scores[art["id"]] = scores.get(art["id"], 0) + 1.0 / (k + rang)
    meilleurs = sorted(scores.items(), key=lambda x: x[1], reverse=True)[n:]
    index = {a["id"]: a for a in liste_vect + liste_lex}
    return [(index[i], s) for i, s in meilleurs]
```

C.3 — Le script de test manuel

```
# python -m ingestion.testeur_recherche "délai de convocation AG SARL"
import sys
from app.services.recherche import rechercher
question = " ".join(sys.argv[1:])
for i, (art, score) in enumerate(rechercher(question), 1):
    print(f"[{i}] score={score:.4f} Article {art['numero']}")
    print(f"      {art['chemin']}")
    print(f"      {art['contenu'][:220]}...")
```

Lance ce script sur une trentaine de questions que tu connais. Pour chacune, l'article attendu doit apparaître **dans les 3 premiers résultats**. Si ce n'est pas le cas, le problème est en amont : découpage trop grossier, chemin hiérarchique absent, ou préfixe manquant à la vectorisation.

C.4 — Calibrer le seuil de refus

Le seuil détermine quand l'assistant refuse de répondre. Trop bas, il répond n'importe quoi ; trop haut, il refuse des questions légitimes. Méthode : passe 20 questions couvertes et 10 questions clairement hors corpus, note le meilleur

score obtenu à chaque fois, et place le seuil entre les deux nuages de points.

Ce seuil se calibre avec des données, jamais au jugement. Note les scores dans un tableau et regarde où se séparent les deux groupes. C'est aussi un excellent élément à montrer au jury : cela prouve une démarche expérimentale.

C.5 — Résultat vérifiable de la phase C

- Sur 30 questions de contrôle, l'article attendu est dans le top 3 au moins 25 fois.
- Les questions hors corpus obtiennent toutes un score inférieur au seuil retenu.
- Le seuil est justifié par un tableau de scores, pas par une intuition.

D. Le jeu d'évaluation

Phase D — 2 à 3 jours. À écrire AVANT d'optimiser le pipeline, jamais après.

Pourquoi maintenant : si tu écris ces questions après avoir réglé le système, tu les écriras inconsciemment pour qu'elles réussissent. Écris maintenant, elles mesurent la réalité. C'est aussi ton meilleur argument devant un jury — tu es le seul candidat capable de *chiffrer* sa fiabilité.

D.1 — Composition des 50 questions

Type	Nombre	Nature
Factuelles simples	25	Un délai, un taux, un nombre minimum d'associés — réponse dans un seul article
Multi-articles	10	Réponse nécessitant deux ou trois articles combinés
Formulations indirectes	5	Question posée comme un vrai utilisateur la poserait, sans vocabulaire juridique
Pièges hors corpus	5	Question de droit du travail ou de droit pénal — l'assistant DOIT refuser
Pièges d'actualité	5	Question portant sur un texte non ingéré — refus attendu également

D.2 — Format du fichier

```
// evaluation/questions.json
[
  {
    "id": 1,
    "question": "Quel est le nombre minimum d'associés dans une SARL ?",
    "type": "factuelle",
    "articles_attendus": ["309"],
    "reponse_attendue": "Un seul associé suffit (SARL unipersonnelle possible).",
    "doit_refuser": false
  },
  {
    "id": 46,
    "question": "Quelle est la durée légale du préavis de licenciement ?",
    "type": "piege_hors_corpus",
    "articles_attendus": [],
    "reponse_attendue": "",
    "doit_refuser": true
  }
]
```

D.3 — Le script d'évaluation

```
# evaluation/lancer_eval.py
import json, httpx
questions = json.load(open("evaluation/questions.json", encoding="utf-8"))
ok_citations = ok_refus = total_refus = 0
for q in questions:
    r = httpx.post("http://localhost:8000/chat/question",
                  json={"question": q["question"]}, timeout=60).json()
    a_refuse = r.get("confiance") == "insuffisante" or r.get("refus") is True
```

```

    if q["doit_refuser"]:
        total_refus += 1
        if a_refuse:
            ok_refus += 1
        else:
            print(f"[{q['id']}] AURAIT DU REFUSER -> {r['reponse'][:90]}")
    else:
        cites = {str(c["numero"]) for c in r.get("citations", [])}
        attendus = set(q["articles_attendus"])
        if attendus & cites:
            ok_citations += 1
        else:
            print(f"[{q['id']}] attendu {attendus}, cite {cites}")
n = len(questions) - total_refus
print(f"Citations correctes : {ok_citations}/{n} ({100*ok_citations/n:.0f} %)")
print(f"Refus corrects      : {ok_refus}/{total_refus}")

```

Rejoue ce script à **chaque modification du pipeline** : changement de seuil, de prompt, de découpage, de modèle. C'est ton filet de sécurité contre les régressions, et le chiffre que tu affiches en Demo Day.

E. L'API : le pipeline RAG complet

Phase E — 1 à 2 semaines. Le cœur du produit, testable sans aucune interface.

E.1 — Le prompt système

C'est le texte qui contraint le modèle. Chaque phrase y a une raison d'être ; ne le raccourcis pas pour économiser des tokens.

```

PROMPT_SYSTÈME = '''Tu es un assistant de recherche documentaire juridique
specialise dans le droit OHADA et la fiscalite camerounaise.
REGLES ABSOLUES
1. Tu reponds UNIQUEMENT a partir des articles fournis ci-dessous.
2. Tu ne cites que des articles presents dans ces extraits, avec leur identifiant exact.
3. Si les extraits ne permettent pas de repondre, tu mets "confiance": "insuffisante"
   et tu expliques en une phrase ce qui manque. Tu n'inventes JAMAIS.
4. Tu ne fais aucune supposition sur des textes absents des extraits.
5. Tu distingues ce que dit le texte de ce qui releve de l'interpretation.
6. Tu reponds en francais, de maniere precise et sobre.
FORMAT DE SORTIE - JSON strict, sans texte autour :
{
  "reponse": "...",
  "citations": [{"article_id": 123, "extrait": "...", "pourquoi": "..."}],
  "confiance": "elevee | moyenne | insuffisante",
  "mise_en_garde": "...
}
'''

```

La question de l'utilisateur ne doit **JAMAIS** être concaténée au prompt système. Elle est envoyée comme message utilisateur séparé, et les articles sont délimités explicitement. Sinon, un utilisateur peut écrire « ignore les instructions précédentes » et contourner toutes tes règles.

E.2 — L'orchestration

```

# app/services/rag.py
from app.services.recherche import rechercher
from app.services.llm import appeler_llm
from app.config import SEUIL_PERTINENCE, NB_ARTICLES_CONTEXTE
def repondre(question: str) -> dict:
    # 1. recherche
    resultats = rechercher(question, n=NB_ARTICLES_CONTEXTE)
    if not resultats or resultats[0][1] < SEUIL_PERTINENCE:
        return {"reponse": "Cette question depasse les textes actuellement "
                        "disponibles dans ma bibliotheque.",
                "citations": [], "confiance": "insuffisante", "refus": True}
    # 2. contexte numerote
    contexte = "\n\n".join(
        f"[ARTICLE id={a['id']}] {a['sigle']} - Article {a['numero']}\n"
        f"{a['chemin']}\n{a['contenu']}"
        for a, _ in resultats)
    # 3. appel LLM
    brut = appeler_llm(

```



```

    systeme=PROMPT_SYSTEME,
    utilisateur=f"ARTICLES DISPONIBLES :\n{contexte}\n\nQUESTION : {question}")
# 4. VALIDATION DES CITATIONS - la garantie du produit
ids_autorises = {a["id"] for a, _ in resultats}
citations = [c for c in brut.get("citations", [])
              if c.get("article_id") in ids_autorises]
if brut.get("citations") and not citations:
    return {"reponse": "Je ne parviens pas a fonder une reponse fiable sur "
                      "les textes disponibles.",
            "citations": [], "confiance": "insuffisante", "refus": True}
brut["citations"] = citations
return brut

```

L'étape 4 est la pièce maîtresse. Le modèle ne peut citer que ce qu'on lui a donné ; toute citation d'un identifiant absent fait rejeter la réponse entière. C'est mécanique, pas déclaratif — c'est ce qui rend ta promesse « zéro réponse inventée » démontrable.

E.3 — Isoler le fournisseur LLM

Tout appel passe par une seule fonction. Si le fournisseur change ses prix ou ses conditions, tu modifies un fichier et rien d'autre.

```

# app/services/llm.py
import httpx, json
from app.config import LLM_API_KEY, LLM_MODELE, URL_FOURNISSEUR
def appeler_llm(systeme: str, utilisateur: str) -> dict:
    reponse = httpx.post(
        URL_FOURNISSEUR,
        headers={"x-api-key": LLM_API_KEY, "content-type": "application/json"},
        json={"model": LLM_MODELE, "max_tokens": 1200, "temperature": 0.1,
            "system": systeme,
            "messages": [{"role": "user", "content": utilisateur}]},
        timeout=60)
    reponse.raise_for_status()
    texte = extraire_texte(reponse.json())
    texte = texte.replace("`json", "").replace("`", "").strip()
    try:
        return json.loads(texte)
    except json.JSONDecodeError:
        return {"reponse": "Erreur de format de reponse.", "citations": [],
            "confiance": "insuffisante", "refus": True}

```

E.4 — L'endpoint

```

# app/routers/chat.py
@router.post("/chat/question")
def poser_question(corps: QuestionEntree,
                  utilisateur = Depends(utilisateur_courant),
                  db = Depends(get_db)):
    if utilisateur.quota_restant <= 0:
        raise HTTPException(402, "Quota mensuel epuise")
    resultat = repondre(corps.question)
    # le quota n'est decompte que si le LLM a reellement ete appele
    if not resultat.get("refus"):
        utilisateur.quota_restant -= 1
        db.commit()
    enregistrer_message(db, utilisateur, corps, resultat)
    return resultat

```

Ne décompte pas le quota sur un refus. Un refus ne coûte rien (aucun appel LLM) et pénaliser l'utilisateur pour une question hors corpus donnerait un très mauvais signal sur une fonctionnalité que tu revendiques.

E.5 — Résultat vérifiable de la phase E

- /docs permet de poser une question et de recevoir une réponse sourcée.
- Le script d'évaluation s'exécute et affiche ses taux.
- Une question hors corpus renvoie un refus sans appel LLM (vérifiable au temps de réponse, quasi instantané).
- Aucune citation renvoyée ne correspond à un article absent du contexte.

F. Le frontend Angular

Phase F — 2 semaines. L'interface habille une logique déjà prouvée.

F.1 — Initialisation

```
ng new frontend --standalone --style=scss --routing
cd frontend
ng add @angular/material
ng add @angular/pwa
npm install marked # rendu du markdown des reponses
```

F.2 — Ordre de construction des écrans

Ordre	Écran / composant	Pourquoi ici
1	bloc-base-legale (composant)	C'est la signature visuelle du produit. Construis-le en premier, avec des données en dur, et soigne-le.
2	Page Chat	Le parcours principal : champ de question, fil de messages, intégration du bloc base légale
3	Page Article	Cible des citations ; sans elle, le clic ne mène nulle part
4	Page Bibliothèque	Navigation par sommaire, dépendante de l'API corpus
5	Accueil, Historique, Compte	Confort, aucun risque technique
6	Méthodologie, PWA, export PDF	Finition, sacrificable si le temps manque

F.3 — Le service de chat

```
// core/services/chat.service.ts
@Injectable({ providedIn: 'root' })
export class ChatService {
  private api = inject(ApiService);
  messages = signal<Message[]>([]);
  enCours = signal(false);
  async poser(question: string) {
    this.messages.update(m => [...m, { role: 'user', contenu: question }]);
    this.enCours.set(true);
    try {
      const r = await this.api.post<ReponseChat>('/chat/question', { question });
      this.messages.update(m => [...m, {
        role: 'assistant',
        contenu: r.reponse,
        citations: r.citations,
        confiance: r.confiance
      }]);
    } finally {
      this.enCours.set(false);
    }
  }
}
```

F.4 — Le composant bloc-base-legale

C'est l'élément le plus important de l'interface. Il doit être visuellement distinct du corps de la réponse — fond légèrement teinté, filet doré à gauche, référence de l'article en tête, extrait en dessous, et la référence entière cliquable.

```
<!-- bloc-base-legale.component.html -->
<div class="base-legale" *ngFor="let c of citations">
  <a class="reference" [routerLink]="['/article', c.article_id]">
    {{ c.sigle }} - Article {{ c.numero }}
  </a>
  <blockquote>{{ c.extrait }}</blockquote>
  <p class="pourquoi" *ngIf="c.pourquoi">{{ c.pourquoi }}</p>
</div>
```

F.5 — Points de vigilance

- **Affiche la réponse avant les citations**, jamais l'inverse : les citations n'apparaissent qu'après validation serveur.
- **État de chargement explicite** : une réponse prend jusqu'à 10 secondes, l'utilisateur doit voir qu'il se passe quelque chose.
- **Le refus a son propre rendu**, sobre et non alarmant : c'est une fonctionnalité, pas une erreur.
- **L'avertissement déontologique est permanent**, en pied de chaque réponse.
- **Teste sur un écran de 360 px dès le début**, pas à la fin : ta cible répond depuis son téléphone.

G. Comptes, quota, bibliothèque, historique

Phase G — 1 semaine. Aucun risque technique, mais indispensable à la démonstration.

Élément	Points d'attention
Inscription / connexion	Hachage bcrypt (coût 12), JWT de 30 minutes, intercepteur Angular qui ajoute l'en-tête Authorization automatiquement
Quota	Décompte côté serveur uniquement — jamais dans le navigateur. Réinitialisation mensuelle par date, pas par compteur.
Bibliothèque	Sommaire construit à partir du champ chemin des articles ; mise en cache côté client (le corpus ne change pas pendant une session)
Lecture d'article	Fil d'Ariane complet, navigation article précédent / suivant, affichage de la version et de la date de consolidation
Historique	Liste des conversations, reprise possible ; bouton d'effacement (engagement de confidentialité)

H. Déploiement et finition

Phase H — 3 à 5 jours. Ne jamais laisser cette phase pour la veille du Demo Day.

H.1 — Ordre de déploiement

1. **Base de données** : créer l'instance PostgreSQL managée, activer pgvector, restaurer un export de la base locale.
2. **API** : déployer le backend, renseigner les variables d'environnement, vérifier /docs en ligne.
3. **Frontend** : configurer l'URL de l'API en production, déployer sur Vercel.
4. **CORS** : restreindre l'API au seul domaine du frontend.
5. **Test complet** : refaire tout le parcours depuis un téléphone, sur réseau mobile.

H.2 — Le plan B de la démonstration

L'hébergement gratuit met parfois plusieurs dizaines de secondes à se réveiller après une période d'inactivité — exactement le scénario du Demo Day. Trois protections :

- **Réveiller l'application 15 minutes avant** de passer, en ouvrant le lien depuis ton téléphone.
- **Instance locale Docker Compose prête et testée**, capable de tourner sans internet sauf pour l'appel LLM.
- **Captures d'écran intégrées aux slides** : réponse sourcée, citation ouverte, refus. Elles couvrent même une panne totale.

H.3 — Livrables à vérifier avant de rendre

- Code source sur GitHub, branches main / formateur / apprenant à jour.
- README avec présentation, architecture, captures et instructions de lancement (front et back).
- Lien de démo public fonctionnel, testé depuis un autre appareil.
- Cahier des charges et document d'architecture joints.

- Jeu d'évaluation et son dernier rapport de résultats.
- Fichier `.env.example` présent, `.env` absent du dépôt.

I. Erreurs fréquentes et comment les éviter

Erreur	Conséquence	Prévention
Ingérer plusieurs textes dès le départ	Bugs de découpage multipliés, impossible de savoir lequel pose problème	Un seul texte validé de bout en bout, puis élargir
Commencer par l'interface Angular	Illusion d'avancer ; le risque réel reste intact jusqu'à la fin	Suivre l'ordre des phases A à H
Écrire le jeu d'évaluation après l'optimisation	Questions inconsciemment écrites pour réussir ; mesure sans valeur	Le figer en phase D, avant tout réglage
Concaténer la question au prompt système	Injection de prompt possible : l'utilisateur contourne les règles	Message utilisateur séparé, articles délimités
Faire confiance à l'OCR sans relecture	Numéros d'articles et taux erronés, indétectables ensuite	Relire un échantillon systématiquement
Décompter le quota sur un refus	Pénalise une fonctionnalité revendiquée	Décompte uniquement si le LLM a été appelé
Commiter le fichier <code>.env</code>	Clé d'API révoquée, parfois après utilisation frauduleuse	<code>.gitignore</code> vérifié AVANT le premier commit
Déployer la veille du Demo Day	Panne découverte sans temps pour réagir	Déployer dès la fin de la phase E, en avance

J. Planning condensé sur 8 semaines

Semaine	Phase	Objectif de fin de semaine
1	A + début B	Environnement opérationnel, PDF collecté, extraction du texte réussie
2	B	Découpage validé, contrôles automatiques au vert, 20 articles vérifiés à la main
3	B + C	Corpus chargé, embeddings générés, recherche testée en ligne de commande
4	C + D	Seuil calibré par les données, 50 questions écrites et figées
5	E	Pipeline RAG complet, validation des citations, première mesure d'évaluation
6	E + F	Déploiement précoce de l'API ; bloc base légale et page Chat
7	F + G	Article, bibliothèque, comptes, quota, historique
8	H	PWA, export PDF, déploiement final, répétition du pitch

Si tu prends du retard, ne rogne jamais sur les phases B, C et D. Sacrifie dans cet ordre : export PDF, calculateurs, PWA, bibliothèque, historique, comptes. Une démonstration avec un chat sourcé irréprochable et rien d'autre vaut infiniment mieux qu'une application complète dont les réponses sont douteuses.